

OSSP (Operating Systems and Systems Programming) –25CS2104E

OSSP LAB – SKILL 5

Aim

To implement single-quote handling in a shell-like parser, preserving literal content, preventing variable expansion, storing quoted strings, validating parsing results, and testing different edge cases.

Algorithm

1. Start the program.
2. Read the input command from the user.
3. Check whether the input contains a single quote (').
4. Identify the starting and ending single quotes.
5. Extract the content between the single quotes.
6. Preserve the extracted content exactly as entered.
7. Do not perform variable expansion on the content.
8. Store the quoted content in a string buffer.
9. Validate whether the opening and closing quotes are present.
10. Display the parsed result.
11. Test edge cases such as empty quotes, spaces, and missing closing quotes.
12. Stop the program.

Task 1

```
bhavana@Bhavana:~$ nano single_quotes.c
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#define MAX 200

int main() {
    char input[MAX];
    char output[MAX];

    printf("Enter text: ");
    fgets(input, MAX, stdin);

    input[strcspn(input, "\n")] = '\0';

    int len = strlen(input);

    if (len >= 2 && input[0] == '\'' && input[len - 1] == '\') {
        int j = 0;

        for (int i = 1; i < len - 1; i++) {
            output[j++] = input[i];
        }

        output[j] = '\0';

        printf("\nSingle quoted string detected.\n");
        printf("Literal content: %s\n", output);
        printf("Variable expansion: NOT allowed.\n");
    }
    else {
        printf("\nError: Invalid single-quoted string.\n");
        printf("Use opening and closing single quotes.\n");
    }

    return 0;
}

```

```

bhavana@Bhavana:~$ gcc single_quotes.c -o single_quotes
bhavana@Bhavana:~$ ./single_quotes
Enter text: 'Hello World'

Single quoted string detected.
Literal content: Hello World
Variable expansion: NOT allowed.

```

Task 2

```

bhavana@Bhavana:~$ nano double_quotes.c

```

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 300

int main() {

    char input[MAX];
    char output[MAX];

    printf("Enter text: ");
    fgets(input, MAX, stdin);

    input[strcspn(input, "\n")] = '\0';

    int len = strlen(input);

    if (len < 2 || input[0] != '"' || input[len - 1] != '"') {

        printf("\nError: Invalid double-quoted string.\n");
        printf("Use opening and closing double quotes.\n");

        return 0;
    }

    int j = 0;

    for (int i = 1; i < len - 1; i++) {

        /* Check for variable */
        if (input[i] == '$') {

            char variable[100];
            int k = 0;

            i++;

```

⌂ Help

⌂ Write Out

⌂ Where Is

⌂ Cut

```

        input[i] != '"') {

            variable[k++] = input[i];
            i++;
        }

        variable[k] = '\0';

        char *value = getenv(variable);

        if (value != NULL) {

            for (int x = 0; value[x] != '\0'; x++) {
                output[j++] = value[x];
            }

        } else {
            output[j++] = '$';
            for (int x = 0; variable[x] != '\0'; x++) {
                output[j++] = variable[x];
            }
        }

        i--;
    }
    else {
        output[j++] = input[i];
    }
}
output[j] = '\0';
printf("\nDouble quoted string detected.\n");
printf("Parsed content: %s\n", output);
printf("Spaces are preserved.\n");
printf("Variable expansion is allowed.\n");
return 0;
}

```

```

bhavana@Bhavana:~$ gcc double_quotes.c -o double_quotes
bhavana@Bhavana:~$ ./double_quotes
Enter text: "Hello World"

Double quoted string detected.
Parsed content: Hello World
Spaces are preserved.
Variable expansion is allowed.

```

Result

The programs were successfully implemented and tested in Ubuntu to demonstrate shell-style handling of single and double quoted strings. Single quotes preserved the content literally and prevented variable expansion, while double quotes preserved spaces and allowed environment-variable expansion. The programs also validated quoted strings and handled invalid or unmatched quotes and other edge cases successfully.